

1. Prezentare Generală & Procesul de Gândire

Filosofia de bază din spatele acestei soluții:

În loc să scriu sute de instrucțiuni de tip `if/else` pentru fiecare tehnologie posibilă, am construit un **Motor de Detecție Bazat pe Reguli** inspirat de standardele din industrie precum *Wappalyzer*. Motorul este complet decuplat de reguli. Acesta preia o bază de date `.json` cu tipare Regex și le caută în trei straturi ale răspunsului HTTP:

1. **Headere HTTP** (pentru a detecta serverele de backend, ex: Nginx, Express, PHP).
2. **HTML Brut** (pentru a detecta framework-urile de frontend, CDN-urile și scripturile incluse, ex: React, Shopify CDN).
3. **Cookie-uri** (pentru a detecta urmele specifice sesiunilor, ex: Magento, WooCommerce).

De ce Python?

Am ales Python pentru că este liderul incontestabil în web scraping și ingineria datelor și sunt familiar cu el. Folosirea bibliotecii `pandas` a permis extragerea rapidă a domeniilor din fișierul `.parquet` furnizat, în timp ce `concurrent.futures` a oferit procesare pe mai multe fire de execuție gata de utilizare, pentru a accelera operațiunile de rețea.

2. Arhitectură & Execuție Pas cu Pas

1. **Preluarea Datelor:** Scriptul citește fișierul `part-00000...parquet` folosind `Pandas` și extrage domeniile unice.
 2. **Încărcarea Regulilor:** Analizează fișierul `reguli.json`, care conține tiparele regex pentru tehnologiile vizate.
 3. **Explorare Concurentă (Crawling):** Folosind un `ThreadPoolExecutor` (maxim 15-20 de executanți), scriptul trimite cereri HTTP GET asincrone către domenii. Am utilizat un `User-Agent` personalizat și un timeout de 10 secunde pentru a gestiona elegant serverele lente sau inactive.
 4. **Potrivirea Tiparelor:** Pentru fiecare răspuns reușit, motorul scanează headerele, cookie-urile și corpul HTML brut comparându-le cu regulile Regex. Odată ce este găsită o potrivire, șirul de caractere specific găsit este înregistrat ca **dovadă**.
 5. **Generarea Rezultatelor:** Rezultatele sunt agregate și exportate într-un fișier JSON structurat (`rezultate_scanare.json`).
-

3. Subiecte de Dezbateri

3.1. Care au fost principalele probleme cu implementarea ta actuală și cum le-ai aborda?

Problema 1: Aplicațiile Web Dinamice (SPA - Single Page Applications)

- **Problema:** Multe site-uri moderne (React, Vue, Angular) încarcă un fișier HTML aproape gol și randează conținutul prin JavaScript *după* încărcarea inițială. O simplă cerere `requests.get()` vede doar scheletul gol, ratând potențial tehnologiile încărcate dinamic.
- **Soluția:** Aș integra o soluție de browser de tip "headless" (fără interfață grafică) precum **Playwright** sau **Puppeteer**. În loc să descarce doar HTML-ul, browserul headless execută JavaScript-ul, permițându-ne să inspectăm DOM-ul complet randat și chiar să interceptăm apelurile XHR/API.

Problema 2: Protecțiile Anti-Bot (Cloudflare, Akamai)

- **Problema:** Multe cereri returnează erori 403 Forbidden sau Timeout deoarece bibliotecile HTTP de bază sunt ușor marcate de Web Application Firewalls.
- **Soluția:** Aș implementa un grup rotativ de proxy-uri (proxy-uri rezidențiale) și aș folosi biblioteci concepute pentru a ocoli amprentarea TLS (ex: `curl-cffi` în Python).

3.2. Cum ai scala această soluție pentru milioane de domenii explorate într-un timp util (1-2 luni)?

Rularea `ThreadPoolExecutor` pe o singură mașină nu este viabilă pentru milioane de domenii. Aș trece de la o arhitectură monolitică la o **Arhitectură Distribuită Bazată pe Evenimente**:

1. **Broker de Mesaje (Apache Kafka sau RabbitMQ):** Aș trimite toate cele 2 milioane de domenii într-un topic Kafka sub formă de sarcini individuale.
2. **Muncitori Distribuți (Kubernetes):** Aș implementa sute de noduri "muncitor" ușoare, containerizate. Fiecare muncitor extrage un domeniu din Kafka, efectuează analiza și trimite rezultatul într-un topic de ieșire.
3. **I/O Asincron:** Aș rescrie logica de crawling folosind `asyncio` și `aiohttp` în loc de cererile sincrone din `requests`. Un singur muncitor asincron poate gestiona mii de conexiuni deschise concurrent cu un consum minim de memorie RAM.
4. **Stocare Data Lake:** Rezultatele agregate ar fi salvate direct în Amazon S3 în fișiere `.parquet` partiționate pentru o analiză rapidă de către echipa de vânzări.

3.3. Cum ai descoperi noi tehnologii în viitor?

Crearea manuală a regulilor regex nu este o strategie sustenabilă pe termen lung. Pentru a descoperi automat noi tehnologii:

1. **Utilizarea Seturilor de Date Open-Source:** Aș construi o integrare pentru a prelua și analiza periodic fișierul oficial `technologies.json` din repository-ul de GitHub open-source al Wappalyzer, actualizând automat baza noastră internă de reguli.
 2. **Detecția Anomaliilor în Headere:** Aș crea un pipeline care stochează toate valorile necunoscute găsite în headerele HTTP `X-Powered-By`, `Server` și `Via`. Dacă o valoare necunoscută apare pe sute de domenii diferite, se declanșează o alertă pentru ca echipa de inginerie să o cartografieze ca o tehnologie nouă.
 3. **Analiza Scripturilor cu AI/LLM:** Pentru fișierele JavaScript necartografiate găsite în `<script src= "...">`, am putea transmite numele scripturilor sau blocurile de comentarii către un pipeline LLM pentru a deduce automat dacă aparțin unui instrument SaaS sau de marketing nou apărut.
-

4. Cum să Rulezi Proiectul

1. Instalează dependențele:
``pip install requests pandas pyarrow``
2. Asigură-te că `reguli.json` și fișierul `.parquet` sunt în directorul rădăcină.
3. Rulează crawler-ul:
`python scraper.py`
4. (Opțional) Vezi statisticile agregate:
`python statistici.py`